REVISED TECHNICAL ARCHITECTURE & PROJECT ROADMAP

LinguaLabel — Part 3 Implementation Plan

Revised 5-part engineering sequence: dataset ingestion, hardened annotation workflows, modular ML baseline engine, advisory human-in-the-loop integration, and regression verification.

PROJECT	DOCUMENT VERSION	EXECUTION STRATEGY	TARGET RUNTIME
LinguaLabel Studio	2.0 (Approved Roadmap)	5-Part Phased Rollout	Python 3.14.8 / Streamlit

Architectural Principles & Quality Guardrails

1. Strict Separation of AI & Human Labels
Model outputs are held solely in isolated, advisory session state (`st.session_state.predictions`). Predictions never automatically commit to disk or overwrite human inputs without explicit annotator confirmation.

2. Script Heuristic vs. Statistical Classifier
Unicode Heuristic: Rule-based script check (Devanagari `U+0900..U+097F` vs. Basic Latin `A..Za..z`).
Classifier: Character n-gram TF-IDF + linear estimator for mixed or ambiguous text.

3. Empirical Performance Integrity
No unverified performance claims (e.g., "near-100% accuracy"). All precision, recall, and F1 metrics must be empirically calculated using evaluation harnesses against ground truth corpora.

4. Graceful Degradation & Fallbacks
If ML models or dependencies are unavailable, the application operates cleanly in full manual mode. Predictions with low confidence (< 60%) trigger an explicit `Uncertain` advisory tag.

Part 1: CSV Upload, Preview, Column Mapping & Stable IDs

Data Ingestion Layer Modernization PART 1

Transition data loading from a static, hardcoded file into a flexible ingestion pipeline while retaining `sample_data.csv` as the default built-in seed dataset.

- **Interactive File Ingestion:** Add `st.sidebar.file_uploader("Upload CSV Dataset", type=["csv"])`. Allow users to toggle between custom uploaded datasets and the built-in demo dataset without losing session continuity.
- **Dataset Preview & Text Column Selection:** Render a compact 3–5 row data preview on upload. Provide a dynamic column selector allowing annotators to pick which column represents the text to annotate (auto-detecting `text`, `sentence`, `content`, `utterance`).
- **Encoding & Schema Validation:** Defend against malformed files, empty rows, and encoding mismatches (`utf-8`, `utf-8-sig`, `latin-1`). Verify that the selected text column contains valid string values.
- **Deterministic Stable IDs:** Preserve existing unique `id` columns. If an ID column is absent, empty, or duplicate, generate deterministic, stable IDs (e.g., content hash or stable sequential index) so annotations can be reliably joined and persisted across reruns.
- **Session Persistence Isolation:** Namespace saved annotations per dataset source so uploading a new file never overwrites annotations from other datasets.

Part 2: Complete & Harden Annotation Workflow & Quality Filters

Human Annotation Studio & Triage Queue Hardening PART 2

Verify, complete, and test all manual labeling workflows, navigation safety, multi-attribute filtering, review notes enforcement, and export formatting.

- **Robust Navigation:** Harden boundary conditions for *Previous*, *Next*, *Jump to Sentence*, and *Go to Next Unreviewed* across empty, single-item, or filtered subsets. Auto-save occurs only when current entries pass all validation checks; otherwise, warn the user and remain on the item.
- **Enhanced Triage Filters:** Expand **Dataset Overview** with quick-filter views: *All Records*, *Pending / Unreviewed*, *Completed / Reviewed*, and *Follow-Up Queue* (`Needs second review` , `Ambiguous` , `Skip`). Ensure text search combines cleanly with categorical filters.
- **Review Notes & Validation Enforcement:** Rigorously test the 4 mandatory validation rules: (1) All four schema fields must be selected; (2) Mandatory notes when any field is `Uncertain` ; (3) Marking as `Reviewed — looks correct` is blocked if any label is empty or uncertain; (4) Explanatory notes are mandatory for any triage status (`Needs second review` , `Ambiguous` , `Skip`).
- **CSV Export Verification:** Guarantee UTF-8 with BOM (`utf-8-sig`) encoding across all downloads, ensuring correct Devanagari Hindi character rendering in Excel and pandas, preserving all original columns alongside the 5 standardized annotation columns.

Part 3: Modular ML Engine (Baselines First & Transformer Harness)

Standalone Machine Learning Inference Package PART 3

Construct an isolated, extensible Python package (`ml/`) delivering predictions for Language, Sentiment, and Text Type. No UI code will reside inside this module.

Task	Deterministic / Lexical Baseline	Statistical / N-Gram Model & Transformer Path
Language	**Unicode Script Heuristic:** Evaluates Devanagari codepoints (`U+0900..U+097F`) vs. Basic Latin (`A..Za..z`). Fast, deterministic.	**Character N-Gram TF-IDF** (2–4 chars) + Linear Classifier for transliterated/mixed text. Later: Multilingual MiniLM / XLM-RoBERTa.
Sentiment	**Bilingual Polarity Lexicon:** Curated Hindi Indic sentiment dictionary + negation parser (`नहीं` , `मत` , `ना`). English VADER.	**TF-IDF + Logistic Regression:** Trained on seed bilingual polarity corpora. Later: Evaluated against IndicBERT / fine-tuned transformer.
Text Type	**Syntactic Token Matcher:** Detects interrogatives (`?` , `क्या` , `who`), requests (`कृपया` , `please`), and greetings (`नमस्ते` , `hello`).	**Word N-Gram Intent Classifier:** Disambiguates declarative statements from polite directives. Later: Sequence classification transformer.

- **Uncertainty & Degradation Contract:** When confidence falls below a configured threshold (< 0.60), the prediction outputs `is_uncertain = True` and label `Uncertain` with explanatory text. If packages or weights are uninitialized, `MLEngine.is_available()` returns `False` without throwing exceptions.

Part 4: Human-in-the-Loop UI Integration

Advisory AI Assistance in Annotation Studio PART 4

Seamlessly surface model suggestions in the annotation workspace as an advisory tool that respects annotator agency and existing validation checklists.

- **Advisory Suggestions Card:** Render an "✨ ML Assistance" card directly above or alongside the manual form in the Annotation Workspace.
- **Visual Provenance & Confidence Badges:** Display predicted label, calibrated confidence score (e.g., `88% confidence`), and classification method pill (`Script Heuristic`, `Lexicon Rules`, or `N-Gram Classifier`).
- **Uncertainty Alert Pill:** When a prediction is marked uncertain, display a warning pill: *"⚠️ Low confidence; annotator verification required."*
- **One-Click Selective or Bulk Acceptance:** Provide individual *"Use Suggestion"* buttons adjacent to each selectbox and an *"Accept All Suggestions"* button. Clicking fills the form controls without auto-saving, allowing the human to adjust, provide mandatory notes if needed, and verify compliance.
- **Sidebar Visibility Control:** Add an *"Enable ML Suggestions"* checkbox in the sidebar. If toggled off or if models are uninitialized, the suggestion interface is completely hidden.

Part 5: Automated Tests, Model Evaluation & Regression Checks

Quality Assurance, Testing Suite & Evaluation Harness PART 5

Establish an automated testing suite, empirical model evaluation tools, documentation updates, and regression defenses.

- **Unit & Integration Test Suite (`pytest`):**
  - `tests/test_upload_and_ids.py`: Validates CSV uploads, column selection, delimiter handling, and stable ID generation.
  - `tests/test_workflow.py`: Verifies navigation edge cases, auto-save conditions, validation rules, and UTF-8-SIG export formatting.
  - `tests/test_ml_language.py`: Verifies script heuristic and n-gram classifier on English, Hindi, and mixed-script text.
  - `tests/test_ml_sentiment.py`: Tests sentiment polarity scoring and Hindi negation parsing (`नहीं`, `मत`, `ना`).
  - `tests/test_ml_text_type.py`: Tests classification of statements, questions, requests, and greetings.
  - `tests/test_degradation.py`: Verifies that the app operates smoothly when ML engines are uninitialized or predictions are uncertain.
- **Empirical Model Evaluation CLI (`ml/evaluate.py`):**
  A standalone script that runs models against labeled test data and computes empirical metrics: confusion matrices, Precision, Recall, and Macro-F1 per class. No metrics are fabricated.
- **Documentation Updates:**
  Update `README.md` with custom CSV upload guidance, ML suggestion controls, evaluation commands, and model limitations.

File-by-File Summary of Planned Changes

Target File	Action	Planned Functional Scope & Non-Disruptive Changes
`app.py`	**Modify**	Integrate CSV file upload, column mapping selector, stable ID generation, ML suggestion card, and form populate handlers. Preserve all CSS styling, theme tokens, and existing validation rules.
`requirements.txt`	**Modify**	Add `scikit-learn ⩾ 1.3.0`, `joblib ⩾ 1.3.0`, and `pytest ⩾ 8.0`.
`README.md`	**Modify**	Document CSV upload features, ML assistance workflow, test execution, and architecture.
`ml/base.py` & `engine.py`	**Create**	Define `PredictionResult` dataclass, abstract predictor interface, and unified coordinator with availability checks.
`ml/*_detector.py`	**Create**	Individual modules for Language (script + n-gram), Sentiment (lexicon + linear), and Text Type (syntax + intent).
`ml/lexicons/*.json`	**Create**	Curated polarity and syntactic intent dictionaries for English and Hindi.
`ml/evaluate.py`	**Create**	Evaluation CLI calculating Precision, Recall, Macro-F1, and confusion matrices against ground truth.
`tests/test_*.py`	**Create**	Comprehensive automated test suite covering ingestion, workflow, ML logic, and graceful degradation.

Dependencies & Runtime Assessment

Package	Version Spec	Python 3.14.8 Compatibility & Execution Strategy
`streamlit`	`⩾ 1.36`	Already operational in virtual environment (`v1.66.0`).
`pandas`	`⩾ 2.0`	Already operational in virtual environment (`v3.0.6`).
`scikit-learn`	`⩾ 1.3.0`	Used for TF-IDF and linear models. Pure Python fallback estimators prepared if wheels encounter binary constraints on Python 3.14.
`joblib` / `pytest`	`⩾ 1.3.0` / `⩾ 8.0`	Pure Python / standard test runner; fully compatible with Python 3.14.

Risk Analysis & Mitigation Matrix

Identified Risk	Severity	Technical Mitigation Strategy
Inconsistent CSV Column Schemas	Medium	Interactive text column picker; validation of non-empty content; generation of deterministic IDs.
Accidental Overwrite of Human Input	Critical	Predictions are kept strictly in advisory session state; form values change only on explicit user click.
Uncalibrated Confidence on Out-of-Domain Text	Medium	Configurable uncertainty threshold (< 0.60) forces `Uncertain` label with mandatory explanatory note prompt.
Missing ML Weights or Dependencies	Low	Decoupled engine with `is_available()` check; app defaults cleanly to 100% manual annotation mode.

✓ Non-Breaking Architecture Guarantee

All existing validation checklists, manual selectbox choices, review status triage queues, and CSV exports remain 100% active and uncompromised. The machine learning engine operates strictly as an advisory agent; the human annotator maintains complete authority over the final submitted record.